

SQL

Géraldine Del Mondo

Département Informatique et Technologies de l'information

Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Plan...

- 1 Présentation
- 2 LDD
- 3 LMD
- 4 LMD-Manipulation
 - Insertion
 - Suppression
 - Modifications
- 5 LMD-Recherche
 - Recherche mono-relation
 - Recherche multi-relations
 - Fonctions prédéfinies
 - GROUP BY
 - Questions quantifiées
- 6 Conclusion

Langage de Requête

- Langage de Définition de Données (LDD)
- Langage de Manipulation de Données (LMD)
- Niveaux :
 - formels : algèbre relationnelle, calcul relationnel de tuples ...
 - Orientés utilisateur : SQL (norme), QUEL, Query By Example (QBE)
 - Couplé avec les langages de programmation (Embedded SQL / C, Fortran, ...)

- Fonctionnalités :
 - Définition et Manipulation de données au format relationnel
 - Contrôle des données (intégrité)
 - Contrôle du multi-utilisateurs (transaction)
 - Gestion de la représentation physique
- Pouvoir d'expression
 - Algèbre relationnelle
 - Fonctions (minimum, maximum, moyenne, somme, nombre)
 - Tri
- Philosophie
 - Non-procédural (en théorie)
 - Mélange de l'algèbre relationnelle et du calcul relationnel de tuples
 - Requête = suite d'opération de l'algèbre [+ Fonction] [+ Tri]

- Fonctionnalités :
 - Définition et Manipulation de données au format relationnel
 - Contrôle des données (intégrité)
 - Contrôle du multi-utilisateurs (transaction)
 - Gestion de la représentation physique
- Pouvoir d'expression
 - Algèbre relationnelle
 - Fonctions (minimum, maximum, moyenne, somme, nombre)
 - Tri
- Philosophie
 - Non-procédural (en théorie)
 - Mélange de l'algèbre relationnelle et du calcul relationnel de tuples
 - Requête = suite d'opération de l'algèbre [+ Fonction] [+ Tri]

- Fonctionnalités :
 - Définition et Manipulation de données au format relationnel
 - Contrôle des données (intégrité)
 - Contrôle du multi-utilisateurs (transaction)
 - Gestion de la représentation physique
- Pouvoir d'expression
 - Algèbre relationnelle
 - Fonctions (minimum, maximum, moyenne, somme, nombre)
 - Tri
- Philosophie
 - Non-procédural (en théorie)
 - Mélange de l'algèbre relationnelle et du calcul relationnel de tuples
 - Requête = suite d'opération de l'algèbre [+ Fonction] [+ Tri]

LDD

LDD : Définition et contrôle

- Schéma des données (relation = TABLE)
- Schéma des vues (VIEW)
- Spécification des contraintes d'intégrité
- Spécification des droits utilisateur
- Spécification du placement physique (non normalisé car SGBD dépendant)

Domaines de base

- Entier : INTEGER
- Décimal : DECIMAL (m,n)
- Réel flottant : FLOAT
- Chaîne de caractère : CHAR(longueur)
- Date : DATE
- + des domaines spécifiques à chaque SGBD (non normalisé)

Valeur nulle (null value)

- Valeur d'un attribut inconnue
 - Au moment de l'insertion, et sera (peut être) fournie ultérieurement
 - Inapplicable : ne sera jamais fournie (exemple : nom de jeune fille pour un garçon)
 - Conséquences
 - Ambiguïtés sur les manipulations : sélection, jointure
 - Introduction d'une valeur : **NULL** (exemple : DEGRE = NULL)
- ATTENTION : valeur nulle ne veut pas dire valeur égale à 0 !

Valeur nulle (null value)

- Valeur d'un attribut inconnue
 - Au moment de l'insertion, et sera (peut être) fournie ultérieurement
 - Inapplicable : ne sera jamais fournie (exemple : nom de jeune fille pour un garçon)
 - Conséquences
 - Ambiguïtés sur les manipulations : sélection, jointure
 - Introduction d'une valeur : **NULL** (exemple : DEGRE = NULL)
- ATTENTION : valeur nulle ne veut pas dire valeur égale à 0 !

Gestion d'un schéma

- Création simple

EXEMPLE

```
CREATE TABLE VINS (  
  NUM_VIN INTEGER,  
  CRU VARCHAR(20),  
  MILLESIME INTEGER);
```

- Modification

EXEMPLE

```
ALTER TABLE VINS ADD COLUMN DEGRE INTEGER;
```

- Suppression (différent de vider la relation de ses n-uplets)

EXEMPLE

```
DROP TABLE VINS
```

Gestion d'un schéma

- Création simple

VINS	NUM_VIN	CRU	MILLESIME

EXEMPLE

```
CREATE TABLE VINS (
  NUM_VIN INTEGER,
  CRU VARCHAR(20),
  MILLESIME INTEGER);
```

- Modification

EXEMPLE

```
ALTER TABLE VINS ADD COLUMN DEGRE INTEGER;
```

- Suppression (différent de vider la relation de ses n-uplets)

EXEMPLE

```
DROP TABLE VINS
```

Gestion d'un schéma

- Création simple

VINS	NUM_VIN	CRU	MILLESIME	DEGRE

EXEMPLE

```
CREATE TABLE VINS (
  NUM_VIN INTEGER,
  CRU VARCHAR(20),
  MILLESIME INTEGER);
```

- Modification

EXEMPLE

```
ALTER TABLE VINS ADD COLUMN DEGRE INTEGER;
```

- Suppression (différent de vider la relation de ses n-uplets)

EXEMPLE

```
DROP TABLE VINS
```

Gestion d'un schéma

- Création simple

EXEMPLE

```
CREATE TABLE VINS (  
  NUM_VIN INTEGER,  
  CRU VARCHAR(20),  
  MILLESIME INTEGER);
```

- Modification

EXEMPLE

```
ALTER TABLE VINS ADD COLUMN DEGRE INTEGER;
```

- Suppression (différent de vider la relation de ses n-uplets)

EXEMPLE

```
DROP TABLE VINS
```

Contrainte d'intégrité



Définition CONTRAINTE D'INTÉGRITÉ

Règle définissant la cohérence de données de la base

- SQL 1
 - non nullité des valeurs d'attribut
 - unicité de la valeur d'un attribut (ou groupe d'attributs)
 - valeur par défaut pour un attribut
 - contrainte de domaine
 - clé primaire
 - intégrité référentielle "minimale"

EXEMPLE

```
CREATE TABLE VINS (  
  NUM_VIN INTEGER UNIQUE NOT NULL,  
  CRU VARCHAR (20),  
  MILLESIME INTEGER,  
  DEGRE INTEGER BETWEEN 5 AND 15);
```


LMD

LMD “manipulation” / LMD “recherche”

- Mise à jour : insertion, modification, suppression
 - Un seul n-uplet
 - Plusieurs n-uplets
- Recherche
 - Mono-relation
 - Multi-relations
 - Avec fonction-agrégat

Schéma **EXEMPLE** : Base COOPERATIVE

VINS (V) (NUM_VIN, CRU, MILLESIME, DEGRE)

VITICULTEURS (VT) (NUM_VITICULTEUR, NOM, PRENOM, VILLE)

PRODUCTIONS (P) (VIN, VITICULTEUR)

BUVEURS (B) (NUM_BUVEUR, NOM, PRENOM, VILLE)

COMMANDES (C) (NUM_COMMANDE, DATE, VIN, QUANTITE, BUVEUR)

EXPEDITION (E) (COMMANDE, DATE, QUANTITE)

LMD - MANIPULATION

LMD-Manipulation : Insertion (EXEMPLES)

```
CREATE TABLE VINS
```

```
(NUM_VIN INTEGER,
```

```
CRU VARCHAR(20),
```

```
MILLESIME INTEGER,
```

```
DEGRE INTEGER);
```

VINS	NUM_VIN	CRU	MILLESIME	DEGRE
	100	Jurançon	1979	12
	200	Gamay	NULL	NULL

Insertion

- Un n-uplet

```
INSERT INTO VINS VALUES (100, 'Jurançon', 1979, 12);
```

```
INSERT INTO VINS (NUM_VIN, CRU) VALUES (200, 'Gamay');
```

- Un ensemble de n-uplets
 - Par fichier (COPY FROM)
 - Par requête

```
INSERT INTO BORDEAUX
```

```
SELECT NUM_VIN, MILLESIME, DEGRE
```

```
FROM VINS
```

```
WHERE CRU= 'Bordeaux'
```

*Il faut avoir créé la table
BORDEAUX en amont :*

LMD-Manipulation : Insertion (EXEMPLES)

```
CREATE TABLE VINS
```

```
(NUM_VIN INTEGER,
```

```
CRU VARCHAR(20),
```

```
MILLESIME INTEGER,
```

```
DEGRE INTEGER);
```

VINS	NUM_VIN	CRU	MILLESIME	DEGRE
	100	Jurançon	1979	12
	200	Gamay	NULL	NULL

Insertion

- Un n-uplet

```
INSERT INTO VINS VALUES (100, 'Jurançon', 1979, 12);
```

```
INSERT INTO VINS (NUM_VIN, CRU) VALUES (200, 'Gamay');
```

- Un ensemble de n-uplets
 - Par fichier (COPY FROM)
 - Par requête

```
INSERT INTO BORDEAUX
```

```
SELECT NUM_VIN, MILLESIME, DEGRE
```

```
FROM VINS
```

```
WHERE CRU= 'Bordeaux'
```

*Il faut avoir créé la table
BORDEAUX en amont :*

LMD-Manipulation : Insertion (EXEMPLES)

```
CREATE TABLE VINS
```

```
(NUM_VIN INTEGER,
```

```
CRU VARCHAR(20),
```

```
MILLESIME INTEGER,
```

```
DEGRE INTEGER);
```

VINS	NUM_VIN	CRU	MILLESIME	DEGRE
	100	Jurançon	1979	12
	200	Gamay	NULL	NULL

Insertion

- Un n-uplet

```
INSERT INTO VINS VALUES (100, 'Jurançon', 1979, 12);
```

```
INSERT INTO VINS (NUM_VIN, CRU) VALUES (200, 'Gamay');
```

- Un ensemble de n-uplets
 - Par fichier (COPY FROM)
 - Par requête

```
INSERT INTO BORDEAUX
```

```
SELECT NUM_VIN, MILLESIME, DEGRE
```

```
FROM VINS
```

```
WHERE CRU= 'Bordeaux'
```

*Il faut avoir créé la table
BORDEAUX en amont :*

Suppression (EXEMPLES)

- “Le vin de numéro 150”

```
DELETE FROM VINS WHERE NUM_VIN = 150 ;
```

- “Les vins de degré < 9 ou > 12 ”

```
DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;
```

- “Tous les n-uplets de VINS”

```
DELETE FROM VINS ;
```

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1979	12
150	Gamay	1980	12
200	Bordeaux	1989	13
250	Gamay	2000	13
300	Muscadet	2014	10

Suppression (**EXEMPLES**)

- “Le vin de numéro 150”

DELETE FROM VINS WHERE NUM_VIN = 150 ;

- “Les vins de degré < 9 ou > 12 ”

DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;

- “Tous les n-uplets de VINS”

DELETE FROM VINS ;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1979	12
200	Bordeaux	1989	13
250	Gamay	2000	13
300	Muscadet	2014	10

Suppression (**EXEMPLES**)

- “Le vin de numéro 150”

DELETE FROM VINS WHERE NUM_VIN = 150 ;

- “Les vins de degré < 9 ou > 12 ”

DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;

- “Tous les n-uplets de VINS”

DELETE FROM VINS ;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1979	12
200	Bordeaux	1989	13
250	Gamay	2000	13
300	Muscadet	2014	10

Suppression (EXAMPLES)

- “Le vin de numéro 150”

DELETE FROM VINS WHERE NUM_VIN = 150 ;

- “Les vins de degré < 9 ou > 12 ”

DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;

- “Tous les n-uplets de VINS”

DELETE FROM VINS ;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
100	Jurançon	1979	12
300	Muscadet	2014	10

Suppression (EXAMPLES)

- “Le vin de numéro 150”

DELETE FROM VINS WHERE NUM_VIN = 150 ;

- “Les vins de degré < 9 ou > 12 ”

DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;

- “Tous les n-uplets de VINS”

DELETE FROM VINS ;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
100	Jurançon	1979	12
300	Muscadet	2014	10

Suppression (EXEMPLES)

- “Le vin de numéro 150”

DELETE FROM VINS WHERE NUM_VIN = 150 ;

- “Les vins de degré < 9 ou > 12 ”

DELETE FROM VINS WHERE DEGRE < 9 OR DEGRE > 12 ;

- “Tous les n-uplets de VINS”

DELETE FROM VINS ;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE

Suppression (EXEMPLES)

- “Les commandes passées par Dupond”

```
DELETE FROM COMMANDES
WHERE BUVEUR IN ( SELECT NUM_BUVEUR
                  FROM BUVEURS
                  WHERE NOM = 'Dupond' );
```

COMMANDES

NUM_COMMANDE	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	5	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	12	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
1	Lebon	Juliette	Brest
2	Dupuis	Robert	Paris
3	Dupond	Jean	Rouen

Suppression (EXEMPLES)

- “Les commandes passées par Dupond”

```
DELETE FROM COMMANDES
WHERE BUVEUR IN ( SELECT NUM_BUVEUR
                  FROM BUVEURS
                  WHERE NOM = 'Dupond' );
```

COMMANDES

NUM_COMMANDE	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	5	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	12	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
1	Lebon	Juliette	Brest
2	Dupuis	Robert	Paris
3	Dupond	Jean	Rouen

Suppression (EXEMPLES)

- “Les commandes passées par Dupond”

```
DELETE FROM COMMANDES
WHERE BUVEUR IN ( SELECT NUM_BUVEUR
                  FROM BUVEURS
                  WHERE NOM = 'Dupond' );
```

COMMANDES

NUM_COMMANDE	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	5	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	12	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
1	Lebon	Juliette	Brest
2	Dupuis	Robert	Paris
3	Dupond	Jean	Rouen

Suppression (EXEMPLES)

- “Les commandes passées par Dupond”

```
DELETE FROM COMMANDES
WHERE BUVEUR IN ( SELECT NUM_BUVEUR
                  FROM BUVEURS
                  WHERE NOM = 'Dupond' );
```

COMMANDES

NUM_COMMANDE	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	5	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	12	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
1	Lebon	Juliette	Brest
2	Dupuis	Robert	Paris
3	Dupond	Jean	Rouen

Suppression (EXEMPLES)

- “Les commandes passées par Dupond”

```
DELETE FROM COMMANDES
WHERE BUVEUR IN ( SELECT NUM_BUVEUR
                  FROM BUVEURS
                  WHERE NOM ='Dupond' );
```

COMMANDES

NUM_COMMANDE	DATE	VIN	QUANTITE	BUVEUR
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
1	Lebon	Juliette	Brest
2	Dupuis	Robert	Paris
3	Dupond	Jean	Rouen

Modifications

EXEMPLES

VITICULTEURS

NUM_VITICULTEUR	NOM	PRENOM	VILLE
10	Jabert	Caroline	Paris
50	Manu	Manuel	Lyon
150	Dupas	Eric	Le Mans
200	Durant	Jean	Paris

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

Modifications

EXEMPLES

VITICULTEURS

NUM_VITICULTEUR	NOM	PRENOM	VILLE
10	Jabert	Caroline	Paris
50	Manu	Manuel	Lyon
150	Dupas	Eric	Bordeaux
200	Durant	Jean	Paris

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

Modifications

EXEMPLES

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1979	12
200	Gamay	1982	13
250	Gamay	2000	12

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

Modifications

EXEMPLES

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1979	12
200	Gamay	1982	14.3
250	Gamay	2000	13.2

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

Modifications

EXEMPLES

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

COMMANDES

NUM..	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	5	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	12	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
3	Dupond	Jean	Rouen

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

Modifications

EXEMPLES

- “Mettre à 'Bordeaux' la ville du viticulteur 150”

```
UPDATE VITICULTEURS
SET     VILLE = 'Bordeaux'
WHERE  NUM_VITICULTEUR = 150;
```

- “Augmenter de 10% le degré des Gamay”

```
UPDATE VINS
SET     DEGRE = DEGRE * 1.1
WHERE  CRU = 'Gamay';
```

- “Augmenter de 10 les quantités commandées par 'Dupond' ”

```
UPDATE COMMANDES
SET     QUANTITE = QUANTITE + 10
WHERE  BUVEUR IN (SELECT NUM_BUVEUR
                  FROM    BUVEURS
                  WHERE   NOM = 'Dupond' ;
```

COMMANDES

NUM..	DATE	VIN	QUANTITE	BUVEUR
21	05/01/2013	300	15	3
22	10/02/2013	200	10	1
23	10/02/2013	250	5	2
24	02/05/2014	150	22	3
25	02/03/2015	150	5	2

BUVEURS

NUM_BUVEUR	NOM	PRENOM	VILLE
3	Dupond	Jean	Rouen

LMD - RECHERCHE

Syntaxe générale de recherche

 **Définition** SYNTAXE GÉNÉRALE RECHERCHE

```
SELECT <liste des attributs >  
FROM   <liste des relations >  
WHERE  <liste des critères >;
```

- **SELECT** : opérateur de projection
- **FROM** : opérateur de produit cartésien
- **WHERE** : opérateur de sélection

Union / Différence : approche algébrique

Requête **EXEMPLE** : "Donner les vins de cru Chablis"

```
SELECT NUM_VIN, MILLESIME, DEGRE  
FROM   VINS  
WHERE  CRU = 'Chablis';
```

Recherche d'information **mono-relation** EXEMPLE

```

SELECT  NUM_VIN, MILLESIME, DEGRE
FROM    VINS
WHERE    CRU = 'Chablis';

```

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

Recherche d'information **mono-relation** EXEMPLE

```
SELECT  NUM_VIN, MILLESIME, DEGRE
FROM    VINS
WHERE    CRU = 'Chablis';
```

RÉSULTAT

NUM_VIN	MILLESIME	DEGRE
25	2000	12.5
125	2010	13
225	1999	13

Recherche d'information **mono-relation**

EXEMPLE (SUITE)

- Requête : "Donner **tous** les vins"

```
SELECT *  
FROM VINS ;
```

Recherche d'information **mono-relation**

EXEMPLE (SUITE)

- Requête : "Donner **tous** les vins"

```
SELECT *  
FROM VINS ;
```

RÉSULTAT			
NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

Recherche d'information **mono-relation**

EXEMPLE (SUITE)

- Requête : " Donner les crus des vins de millésime 1976 et de degré 12, triés par ordre croissant"

SELECT CRU

FROM VINS

WHERE MILLESIME = 1976 AND DEGRE = 12

ORDER BY CRU ;

VINS			
NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

RÉSULTAT :

CRU
Gamay
Jurançon

EXEMPLE (SUITE)

- Requête : “Donner la liste de tous les crus (→ **sans double**)”

SELECT DISTINCT CRU
FROM VINS;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

Recherche d'information **mono-relation**

EXEMPLE (SUITE)

- Requête : “Donner la liste de tous les crus (→ **sans double**)”

SELECT DISTINCT CRU
FROM VINS;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

RÉSULTAT (**sans distinct**) :

CRU
Jurançon
Chablis
Muscadet
Bordeaux
Bordeaux
Chablis
Muscadet
Gamay
Bordeaux
Chablis
Gamay

EXEMPLE (SUITE)

- Requête : “Donner la liste de tous les crus (→ **sans double**)”

SELECT DISTINCT CRU
FROM VINS;

VINS

NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

RÉSULTAT (**avec distinct**) :

CRU
Gamay
Chablis
Muscadet
Jurançon
Bordeaux

Recherche d'information **mono-relation** : sur un **intervalle de valeurs**

EXEMPLES

- Requête : "Vins de degré **compris entre 8 et 12**"

```
SELECT *  
FROM VINS  
WHERE DEGRE >= 8  
      AND DEGRE <= 12;
```

```
SELECT *  
FROM VINS  
WHERE DEGRE BETWEEN 8 AND 12;
```

```
SELECT *  
FROM VINS  
WHERE DEGRE IN (8,9,10,11,12);
```

NUM_VIN	CRU	MILLESIME	DEGRE
25	Chablis	2000	12.5
50	Muscadet	2010	8
75	Bordeaux	2001	13
100	Jurançon	1976	12
125	Chablis	2010	13
150	Muscadet	2014	10
175	Gamay	1976	12
200	Bordeaux	1989	13
225	Chablis	1999	13
250	Gamay	2000	13
275	Bordeaux	1976	14

Recherche d'information **mono-relation** : sur un **intervalle de valeurs**

EXEMPLES

- Requête : “Vins de degré **compris entre 8 et 12**”

```
SELECT *  
FROM VINS  
WHERE DEGRE >= 8  
      AND DEGRE <= 12;
```

NUM_VIN	CRU	MILLESIME	DEGRE
50	Muscadet	2010	8
100	Jurançon	1976	12
150	Muscadet	2014	10
175	Gamay	1976	12

```
SELECT *  
FROM VINS  
WHERE DEGRE BETWEEN 8 AND 12;
```

```
SELECT *  
FROM VINS  
WHERE DEGRE IN (8,9,10,11,12);
```

EXEMPLE

- Requête : “Vins dont le nom du cru commence par 'B' ou 'b'”

```
SELECT *  
FROM VINS  
WHERE CRU LIKE 'B%' OR 'b%'
```

Multi-relations : Expression des jointures

Définition JOINTURE

Jointure = sélection (WHERE) sur produit cartésien (FROM)

EXEMPLE

- “Donner les numéros et noms des viticulteurs produisant du 'Muscadet'”

Sélection :

- CRU = 'Muscadet'
- Numéros de viticulteur identiques dans **VITICULTEURS** et **PRODUCTIONS**
- Numéros de vin identiques dans **VINS** et **PRODUCTIONS**

```
SELECT NUM_VITICULTEUR, NOM  
FROM VITICULTEURS, VINS, PRODUCTIONS  
WHERE NUM_VITICULTEUR = VITICULTEUR AND  
          VIN = NUM_VIN AND  
          CRU = 'Muscadet';
```

Illustration jointure

... **WHERE** NUM_VITICULTEUR = VITICULTEUR...

VITICULTEURS

NUM_VITICULTEUR	NOM	PRENOM	VILLE
10	Jabert	Caroline	Paris
50	Manu	Manuel	Lyon
150	Dupas	Eric	Le Mans
200	Durant	Jean	Paris

PRODUCTIONS

VIN	VITICULTEUR
150	10
25	50
50	150
175	200
250	200
75	200

Illustration jointure

... **WHERE** NUM_VITICULTEUR = VITICULTEUR...

VITICULTEUR ⋈ PRODUCTIONS

NUM_VITICULTEUR	NOM	PRENOM	VILLE	VIN	VITICULTEUR
10	Jabert	Caroline	Paris	150	10
50	Manu	Manuel	Lyon	25	50
150	Dupas	Eric	Le Mans	50	150
200	Durant	Jean	Paris	175	200
200	Durant	Jean	Paris	250	200
200	Durant	Jean	Paris	75	200

Illustration jointure

... **AND** VIN = NUM_VIN...

VITICULTEUR ⋈ PRODUCTIONS

NUM_VITICULTEUR	NOM	PRENOM	VILLE	VIN	VITICULTEUR
10	Jabert	Caroline	Paris	150	10
50	Manu	Manuel	Lyon	25	50
150	Dupas	Eric	Le Mans	50	150
200	Durant	Jean	Paris	175	200
200	Durant	Jean	Paris	250	200
200	Durant	Jean	Paris	75	200

VINS	NUM_VIN	CRU	MILLESIME	DEGRE
	25	Chablis	2000	12.5
	50	Muscadet	2010	8
	75	Bordeaux	2001	13
	100	Jurançon	1976	12
	125	Chablis	2010	13
	150	Muscadet	2014	10
	175	Gamay	1976	12
	200	Bordeaux	1989	13
	225	Chablis	1999	13
	250	Gamay	2000	13
	275	Bordeaux	1976	14

Illustration jointure

... **AND** VIN = NUM_VIN...

VITICULTEUR⋈PRODUCTIONS⋈VINS

NUM_VIT..	NOM	PRENOM	VILLE	VIN	VITI..	NUM_VIN	CRU	MIL..	DEGRE
10	Jabert	Caroline	Paris	150	10	150	Muscadet	2014	10
50	Manu	Manuel	Lyon	25	50	25	Chablis	2000	12.5
150	Dupas	Eric	Le Mans	50	150	50	Muscadet	2010	8
200	Durant	Jean	Paris	175	200	175	Gamay	1976	12
200	Durant	Jean	Paris	250	200	250	Gamay	2000	13
200	Durant	Jean	Paris	75	200	75	Bordeaux	2001	13

Illustration jointure

... **AND** CRU = 'Muscadet' ;

VITICULTEUR⋈PRODUCTIONS⋈VINS

NUM_VIT..	NOM	PRENOM	VILLE	VIN	VITI..	NUM_VIN	CRU	MIL..	DEGRE
10	Jabert	Caroline	Paris	150	10	150	Muscadet	2014	10
150	Dupas	Eric	Le Mans	50	150	50	Muscadet	2010	8

Illustration jointure

SELECT NUM_VITICULTEUR, NOM ...

RÉSULTAT :

NUM_VITICULTEUR	NOM
10	Jabert
150	Dupas

Attention ! L'ordre d'exécution des opérations dépend de l'optimiseur de requête.

Blocs imbriqués

Jointure de manière procédurale : [EXEMPLE](#)

```
SELECT NUM_VITICULTEUR, NOM
FROM    VITICULTEURS
WHERE    NUM_VITICULTEUR IN
            SELECT VITICULTEUR
            FROM    PRODUCTIONS
            WHERE    VIN IN
                    SELECT NUM_VIN
                    FROM    VINS
                    WHERE    CRU = 'Muscadet' ;
```

Auto-Jointure (1)



Définition AUTO-JOINTURE

Jointure d'une relation avec elle-même

EXEMPLE “Donner les couples de buveurs habitant la même ville”

	NUM_BUVEUR	VILLE
t_1	1	Paris
t_2	2	Nantes
t_3	3	Paris
...	...	...

Les buveurs parisiens :

```
SELECT NUM_BUVEUR
FROM BUVEURS
WHERE VILLE = 'Paris' ;
```

Est-ce que le tuple t_1 répond au critère ?

Est-ce que le tuple t_2 répond au critère ?

...

Auto-Jointure (1)



Définition AUTO-JOINTURE

Jointure d'une relation avec elle-même

EXEMPLE “Donner les couples de buveurs habitant la même ville”

	NUM_BUVEUR	VILLE
t_1	1	Paris
t_2	2	Nantes
t_3	3	Paris
...	...	...

ce qu'on aurait éventuellement envie d'écrire ... :

```
SELECT NUM_BUVEUR B1, NUM_BUVEUR B2
FROM BUVEURS
WHERE B1.VILLE = B2.VILLE;
```

... **mais qui est faux !**

Comment accéder à deux numéros de buveurs distincts sur un tuple ?

Auto-Jointure (2)

- “Donner les couples de buveurs habitant la même ville”

```

SELECT B1.NUM_BUVEUR, B1.VILLE , B2.NUM_BUVEUR
FROM    BUVEURS B1, BUVEURS B2
WHERE   B1.VILLE = B2.VILLE AND
          B1.NUM_BUVEUR > B2.NUM_BUVEUR;
  
```

BUVEUR(B1)	NUM_BUVEUR	VILLE
t_1	1	Paris
t_2	2	Nantes
t_3	3	Paris
...	...	...

BUVEUR(B2)	NUM_BUVEUR	VILLE
t_1	1	Paris
t_2	2	Nantes
t_3	3	Paris
...	...	...

Opérateurs ensemblistes (**EXEMPLE**)



Définition OPÉRATEURS ENSEMBLISTES

Opérateurs binaires avec deux relations de même schéma en entrée (élimination automatique des doubles en sortie)

- **UNION**

```
(SELECT NUM_VITICULTEUR FROM VITICULTEURS)
```

```
UNION
```

```
(SELECT NUM_BUVEUR FROM BUVEURS)
```

- **INTERSECT**

```
(SELECT NUM_VITICULTEUR FROM VITICULTEURS)
```

```
INTERSECT
```

```
(SELECT NUM_BUVEUR FROM BUVEURS)
```

- **EXCEPT**

```
(SELECT NUM_VITICULTEUR FROM VITICULTEURS)
```

```
EXCEPT
```

```
(SELECT NUM_BUVEUR FROM BUVEURS)
```

Fonctions prédéfinies



Principe FONCTIONS PRÉDÉFINIES

- COUNT, SUM, AVG, MIN, MAX
- Elles s'appliquent à l'ensemble des valeurs d'**UNE** colonne d'une relation et produit une valeur unique

Cas particulier : COUNT(*) : nombre de n-uplets

EXEMPLE

- “Donner le nombre de n-uplets de **VINS**”

```
SELECT COUNT(*)  
FROM VINS;
```

Fonctions prédéfinies : Positionnement dans le **SELECT**

EXEMPLES

- “Donner la **moyenne** des degrés de tous les vins”

```
SELECT AVG(DEGRE)  
FROM VINS ;
```

- “Donner la **quantité totale** commandée par le buveur de nom "Grosbuveur"”

```
SELECT SUM(QUANTITE)  
FROM COMMANDES, BUVEURS  
WHERE NOM = 'Grosbuveur' AND  
        NUM_BUVEUR = BUVEUR ;
```

Fonctions prédéfinies : Positionnement dans le **WHERE**

EXEMPLES

- “Donner les vins dont le degré est supérieur à la moyenne des degrés de tous les vins”

```
SELECT  *  
FROM    VINS  
WHERE   DEGRE > (SELECT AVG(DEGRE)  
                  FROM    VINS);
```

- “Donner les numéros de commande où la quantité commandée a été totalement expédiée”

```
SELECT NUM_COMMANDE  
FROM COMMANDES C  
WHERE C.QUANTITE = (SELECT SUM(E.QUANTITE)  
                    FROM EXPEDITIONS E  
                    WHERE E.COMMANDE = C.NUM_COMMANDE);
```

Illustration

Commandes (C)	num_commande	date	vin	quantite	buveur
	1	2/07/20	56	100	4
	2	5/09/20	56	50	8
	3	15/11/20	3	5	3
	...	...			

C.num_commande

C.quantite

Expeditions (E)	commande	date	quantite
	1	2/07/20	50
	1	3/07/20	25
	2	10/09/20	25
	2	15/09/20	25
	3	15/11/20	5

E.commande

= ?

$\sum E.quantite$

Agrégats

Fonctions de calcul :

- Somme (SUM), moyenne (AVG)
- Minimum (MIN), maximum (MAX)
- Comptage (COUNT)

1. "Moyenne des degrés des VINS disponibles" $\Rightarrow$ 1 relation, 1 attribut, 1 n-uplet
2. "Moyenne des degrés des VINS par CRU" : ?

→ Partition horizontale d'une relation

EXEMPLE

VINS	NUMERO_VIN	CRU	MILLESIME	DEGRE	QUANTITE
	3	Chablis	1977	10,9	100
	5	Volnay	1977	10,8	400
	6	Chablis	1987	11,9	250
	8	Medoc	1985	11,2	200
	7	Volnay	1986	11,2	300

- “Moyenne des degrés des vins disponibles”

- Résultat : 1 relation, 1 attribut, 1 n-uplet

- $R = \text{AVG}(\mathbf{VINS}, \text{DEGRE})$
- | | |
|---|------|
| R | AVG |
| | 11,2 |

Insuffisance des fonctions

VINS	NUMERO_VIN	CRU	MILLESIME	DEGRE	QUANTITE
	3	Chablis	1977	10,9	100
	5	Volnay	1977	10,8	400
	6	Chablis	1987	11,9	250
	8	Medoc	1985	11,2	200
	7	Volnay	1986	11,2	300

- Somme des quantités par cru
 - Résultat : 1 relation, 2 attributs, 1 ou plusieurs n-uplet(s)

- $R = \text{SUM}(\mathbf{VINS}, \text{CRU}, \text{QUANTITE})$

R	CRU	QUANTITE
	Chablis	350
	Volnay	700
	Medoc	200

→ Fonctions sur les groupes

EXEMPLES

- “Donner, pour chaque cru, la somme des quantités des vins de ce cru”

```
SELECT CRU, SUM(QUANTITE)
FROM VINS
GROUP BY CRU ;
```

- Idem “avec tri par degré moyen décroissant”

```
SELECT    CRU, AVG(QUANTITE)
FROM      VINS
GROUP BY  CRU
ORDER BY  2 DESC ;
```

Partition d'une relation - **GROUP BY**



Principe GROUP BY

- Partition horizontale d'une relation, selon les valeurs d'un attribut (ou d'un groupe d'attributs), spécifié dans la clause **GROUP BY**
- La relation est (logiquement) fragmentée en groupe de n-uplets, où tous les n-uplets de chaque groupe ont la même valeur pour l'attribut (ou le groupe d'attributs) de partition
- Application possible d'une fonction à chaque groupe
- Restriction sur les groupes en fonction de critères (Clause **HAVING**)

Requête erronée

```
SELECT CRU, NUM_VIN, AVG(DEGRE)  
FROM VINS  
GROUP BY CRU ;
```

→ Résultats "attendus" :

CRU	NUM_VIN	AVG(DEGRE)
Bordeaux	{1, 3, 6, 10}	10.0
Chablis	{5, 7}	11.0
Jurançon	{2, 8, 11}	13.0

→ Requête non valide en SQL

- NUM_VIN est multi-valué par rapport à CRU
- La relation n'est pas en 1FN (→ cours Normalisation)
- En pratique : Il faut **toujours** mettre tous les attributs du **SELECT** (sauf agrégat) dans le **GROUP BY**.

→ Restriction sur les groupes (1)



RESTRICTION

- Clause **WHERE** : sur les n-uplets d'une relation
- Clause **HAVING** : sur les groupes obtenus par un **GROUP BY**

EXEMPLE

- Exemple : " Donner les numéros et les noms des buveurs **ayant commandé plus d'une fois**, ainsi que la moyennes des quantités commandées

NUM.C...	DATE	VIN	QUANTITE	BUVEUR	NOM	PRENOM	VILLE
21	05/01/2013	300	5	3	Dupond	Jean	Rouen
22	10/02/2013	200	10	1	Durant	Martine	Paris
23	10/02/2013	250	5	2	Magritte	Emile	Nantes
24	02/05/2014	150	12	3	Dupond	Jean	Rouen
25	02/03/2015	150	5	2	Magritte	Emile	Nantes

→ Restriction sur les groupes (2)

GROUP BY...

NUM_C..	DATE	VIN	QUANTITE	BUVEUR	NOM	PRENOM	VILLE
21	05/01/2013	300	5	3	Dupond	Jean	Rouen
24	02/05/2014	150	12	3	Dupond	Jean	Rouen
22	10/02/2013	200	10	1	Durant	Martine	Paris
23	10/02/2013	250	5	2	Magritte	Emile	Nantes
25	02/03/2015	150	5	2	Magritte	Emile	Nantes

HAVING...

NUM_C..	DATE	VIN	QUANTITE	BUVEUR	NOM	PRENOM	VILLE
21	05/01/2013	300	5	3	Dupond	Jean	Rouen
24	02/05/2014	150	12	3	Dupond	Jean	Rouen
23	10/02/2013	250	5	2	Magritte	Emile	Nantes
25	02/03/2015	150	5	2	Magritte	Emile	Nantes

```

SELECT NUM_BUVEUR, NOM, AVG(QUANTITE)
FROM BUVEURS, COMMANDES
WHERE NUM_BUVEUR = BUVEUR
GROUP BY NUM_BUVEUR, NOM
HAVING COUNT(*) > 1;

```

BUVEUR	NOM	AVG(QUANTITE)
3	Dupond	8.5
2	Magritte	5

Questions quantifiées (**ALL**, **ANY**, **EXISTS**)

Définition ALL

Teste si la valeur d'un attribut satisfait un critère de comparaison avec **TOUS** les résultats d'une sous-requête

Définition ANY

Teste si la valeur d'un attribut satisfait un critère de comparaison avec **AU MOINS** un résultat d'une sous-requête

EXEMPLE

- “Donner les noms et numéros des buveurs qui ont le plus commandé”

```
SELECT NUM_BUVEURS, NOM
FROM   BUVEURS, COMMANDES
WHERE  NUM_BUVEUR = C.BUVEUR AND
        QUANTITE >= ALL ( SELECT QUANTITE
                           FROM   COMMANDES );
```

Questions quantifiées **EXEMPLES**

(requête principale)

(sous-requête)

ALL

NUM_C..	DATE	VIN	QUANT.	BUVEUR	NOM	PRENOM	VILLE	QUANTITE
21	05/01/2013	300	5	3	Dupond	Jean	Rouen	5
22	10/02/2013	200	10	1	Durant	Martine	Paris	10
23	10/02/2013	250	5	2	Magritte	Emile	Nantes	5
24	02/05/2014	150	12	3	Dupond	Jean	Rouen	12
25	02/03/2015	150	4	2	Magritte	Emile	Nantes	4

ANY

NUM_C..	DATE	VIN	QUANT.	BUVEUR	NOM	PRENOM	VILLE	QUANTITE
21	05/01/2013	300	5	3	Dupond	Jean	Rouen	5
22	10/02/2013	200	10	1	Durant	Martine	Paris	10
23	10/02/2013	250	5	2	Magritte	Emile	Nantes	5
24	02/05/2014	150	12	3	Dupond	Jean	Rouen	12
25	02/03/2015	150	4	2	Magritte	Emile	Nantes	4

Prédicat d'existence (**EXISTS**)

Définition EXISTS

Teste si la réponse à une sous-requête est vide ou non

EXEMPLE

- “Viticulteurs ayant produit au moins un vin (quel que soit ce vin)”

```
SELECT VT.*  
FROM   VITICULTEURS VT  
WHERE  EXISTS ( SELECT P.*  
                FROM   PRODUCTIONS P  
                WHERE  VT.NUM_VITICULTEUR =  
                      P.VITICULTEUR);
```


Division via EXISTS

- Quels sont les viticulteurs ayant produit TOUS les vins ?
- ➔ Paraphrase : Un viticulteur est sélectionné s'il n'existe aucun vin qui n'ait pas été produit par ce viticulteur (double négation)

```
SELECT VT.*
FROM VITICULTEURS VT
WHERE NOT EXISTS (
    SELECT V.*
    FROM VINS V
    WHERE NOT EXISTS (
        SELECT P.*
        FROM PRODUCTIONS P
        WHERE P.VITICULTEUR = VT.NUM_VITICULTEUR
        AND P.VIN = V.NUM_VIN;
```

Division via EXISTS

- Quels sont les viticulteurs ayant produit TOUS les vins ?
- Paraphrase : Un viticulteur est sélectionné s'il n'existe aucun vin qui n'ait pas été produit par ce viticulteur (double négation)

```
SELECT VT.*  
FROM VITICULTEURS VT  
WHERE NOT EXISTS (  
    SELECT V.*  
    FROM VINS V  
    WHERE NOT EXISTS (  
        SELECT P.*  
        FROM PRODUCTIONS P  
        WHERE P.VITICULTEUR = VT.NUM_VITICULTEUR  
        AND P.VIN = V.NUM_VIN;
```

Division via EXISTS

- Quels sont les viticulteurs ayant produit TOUS les vins ?
- Paraphrase : Un viticulteur est sélectionné s'il n'existe aucun vin qui n'ait pas été produit par ce viticulteur (double négation)

```
SELECT VT.*  
FROM VITICULTEURS VT  
WHERE NOT EXISTS (  
    SELECT V.*  
    FROM VINS V  
    WHERE NOT EXISTS (  
        SELECT P.*  
        FROM PRODUCTIONS P  
        WHERE P.VITICULTEUR = VT.NUM_VITICULTEUR  
        AND P.VIN = V.NUM_VIN;
```

Division via EXISTS

- Quels sont les viticulteurs ayant produit TOUS les vins ?
- Paraphrase : Un viticulteur est sélectionné s'il n'existe aucun vin qui n'ait pas été produit par ce viticulteur (double négation)

```
SELECT VT.*  
FROM VITICULTEURS VT  
WHERE NOT EXISTS (  
    SELECT V.*  
    FROM VINS V  
    WHERE NOT EXISTS (  
        SELECT P.*  
        FROM PRODUCTIONS P  
        WHERE P.VITICULTEUR = VT.NUM_VITICULTEUR  
        AND P.VIN = V.NUM_VIN ;
```

La division : “intuition” pour la requête SQL

- Que veut-on ?
 - Les viticulteurs → Requête principale
- Sur quoi porte la division ?
 - Les vins → Sous-requête 1
- Qu'est-ce qui les relie ?
 - La production → Sous-requête 2

Résumé conditions de recherche

- Sélection de n-uplets (**WHERE**),
sélection de groupes (**HAVING**)
- Conjonction/Disjonction/Négation de triplet (Attribut Comparateur, Valeur)
- Expression évaluée à Vrai / Faux / Inconnu
- Condition élémentaire (prédicat)
 - Comparaison : =, <, <=, ... entre attributs ou attribut / valeur
 - **BETWEEN, LIKE, IN, IS NULL, EXISTS, ANY, ALL**

Résumé Requête de recherche SQL

- (6) **SELECT** <attributs et/ou expressions sur attributs> A_j
- (1) **FROM** <relations> R_i
- (2) **WHERE** <conditions sur les n-uplets> C_i
- (3) **GROUP BY** <attributs> A_k
- (4) **HAVING** <conditions sur les groupes> C_j
- (5) **ORDER BY** <attributs> A_l

Interprétation :

- (1) Produit Cartésien des relations R_i
- (2) Sélection des n-uplets de (1) vérifiant C_i
- (3) Partition de l'ensemble obtenu suivant les valeurs A_k
- (4) Sélection des groupes de (3) vérifiant C_j
- (5) Tri des groupes obtenus en (4) suivant les valeurs A_l
- (6) π de (5) sur A_j , et fonctions sur les groupes (s'il y en a)

EXEMPLE complet

- Donnez par ordre croissant le cru des vins commandés par les buveurs parisiens, ainsi que la somme des quantités commandées, uniquement si cette quantité est strictement supérieure à 100 litres

VINS (V) (NUM_VIN, CRU, MILLESIME, DEGRE)

VITICULTEURS (VT) (NUM_VITICULTEUR, NOM, PRENOM, VILLE)

PRODUCTIONS (P) (VIN, VITICULTEUR)

BUVEURS (B) (NUM_BUVEUR, NOM, PRENOM, VILLE)

COMMANDES (C) (NUM_COMMANDE, DATE, VIN, QUANTITE, BUVEUR)

EXPEDITION (E) (COMMANDE, DATE, QUANTITE)

EXEMPLE complet

- Donnez par ordre croissant le cru des vins commandés par les buveurs parisiens, ainsi que la somme des quantités commandées, uniquement si cette quantité est strictement supérieure à 100 litres

```
SELECT    CRU, SUM(QUANTITE)
FROM      BUVEURS, COMMANDES, VINS
WHERE     NUM_BUVEUR = BUVEUR AND
          VIN = NUM_VIN AND
          VILLE = 'Paris'

GROUP BY  CRU
HAVING    SUM(QUANTITE) > 100
ORDER BY  CRU;
```

CONCLUSION

Conclusion

- LDD (Administrateur)
- LMD (Développeur / Utilisateur)

→ LDD

- Création - Suppression - Modification vis à vis du **schéma** de la base

→ LMD

- Manipulation : Insertion - Suppression - Modification de tuples
- Recherche : mono/multi-relation(s) - Jointure - opérateurs ens. - Agrégats - Group By - questions quantifiées

Conclusion

- LDD (Administrateur)
- LMD (Développeur / Utilisateur)

→ LDD

- Création - Suppression - Modification vis à vis du **schéma** de la base

→ LMD

- Manipulation : Insertion - Suppression - Modification de tuples
- Recherche : mono/multi-relation(s) - Jointure - opérateurs ens. - Agrégats - Group By - questions quantifiées

Conclusion

- LDD (Administrateur)
- LMD (Développeur / Utilisateur)

→ LDD

- Création - Suppression - Modification vis à vis du **schéma** de la base

→ LMD

- Manipulation : Insertion - Suppression - Modification de tuples
- Recherche : mono/multi-relation(s) - Jointure - opérateurs ens. - Agrégats - Group By - questions quantifiées

Conclusion

- LDD (Administrateur)
- LMD (Développeur / Utilisateur)

→ LDD

- Création - Suppression - Modification vis à vis du **schéma** de la base

→ LMD

- Manipulation : Insertion - Suppression - Modification de tuples
- Recherche : mono/multi-relation(s) - Jointure - opérateurs ens. - Agrégats - Group By - questions quantifiées

Conclusion

- LDD (ADMINISTRATEUR)
- LMD (Développeur / Utilisateur)

→ LDD

- Création - Suppression - Modification vis à vis du **schéma** de la base

→ LMD

- Manipulation : Insertion - Suppression - Modification de tuples
- Recherche : mono/multi-relation(s) - Jointure - opérateurs ens. - Agrégats - Group By - questions quantifiées